

TABLE OF CONTENTS

PAC Form	ii
DECLARATION	iii
CERTIFICATE	iv
ACKNOWLEDGEMENT	v
1 INTRODUCTION	1
2 PROFILE OF THE PROBLEM, RATIONALE & SCOPE OF THE STUDY	1
2.1 Profile of the Problem	1
2.2 Rationale	2
2.3 Scope of the Study	3
2.4 Problem Statement	4
3 EXISTING SYSTEM	4
3.1 Introduction	4
3.2 Existing Software Analysis	4
3.2.1 Wazuh / OSSEC	4
3.2.2 Splunk / Elastic Security	5
3.2.3 Commercial EDR Platforms	5
3.2.4 Manual Log Review and Script-Based Monitoring	5
3.3 Summary of Gaps in Existing Systems	6
3.4 Data Flow Diagram: Existing System	6
3.5 Innovations in the Proposed ScropIDS System	6
4 PROBLEM ANALYSIS	7
4.1 Product Definition	7
4.2 Feasibility Analysis	8
4.2.1 Technical Feasibility	8
4.2.2 Economic Feasibility	9

4.2.3	Operational Feasibility	9
4.2.4	Time Feasibility	9
4.2.5	Legal and Ethical Feasibility	10
4.3	Project Plan	11
5	SOFTWARE REQUIREMENT ANALYSIS	11
5.1	Introduction	11
5.2	General Description	12
5.3	Specific Requirements	12
5.3.1	Functional Requirements	12
5.3.2	Non-Functional Requirements	13
6	DESIGN	14
6.1	System Architecture	14
6.2	Design Notations	15
6.2.1	Data Flow Diagrams (DFD)	15
6.2.2	Entity Relationship Diagram	15
6.3	Detailed Module Design	16
6.3.1	Agent Enrollment and Collection Module	16
6.3.2	Secure Ingest and Tenancy Module	16
6.3.3	Aggregation, Rule Engine, and Threat Analysis Module	16
6.3.4	Alerting, Dashboard, and Notification Module	17
6.4	Flowcharts	18
6.4.1	Agent Enrollment Flowchart	18
6.4.2	Alert Generation Flowchart	19
6.5	Pseudocode	19
6.5.1	Aggregation Algorithm	19
6.5.2	Threat Escalation Algorithm	19
7	TESTING	20
7.1	Functional Testing	20
7.2	Structural Testing (White-Box Testing)	21
7.3	Levels of Testing	22

7.3.1	Unit Testing	22
7.3.2	Integration Testing	22
7.3.3	System Testing	23
7.3.4	User Acceptance Testing (UAT)	23
7.3.5	Performance Testing	23
8	IMPLEMENTATION	24
8.1	Implementation of the Project	24
8.2	Conversion Plan	24
8.3	Post-Implementation and Software Maintenance	25
9	PROJECT LEGACY	26
9.1	Current Status of the Project	26
9.2	Remaining Areas of Concern	26
9.3	Technical and Managerial Lessons Learnt	26
9.3.1	Technical Lessons	26
9.3.2	Managerial Lessons	27
10	USER MANUAL (HELP GUIDE)	27
10.1	Introduction	27
10.2	System Requirements	27
10.3	Dashboard / Analyst User Guide	28
10.3.1	Account Registration and Login	28
10.3.2	Using the Overview and Alerts Workspace	28
10.3.3	Downloading, Enrolling, and Monitoring Agents	28
10.3.4	Handling Alerts and Recommended Actions	28
10.4	Tenant Administrator Guide	29
10.4.1	Organization Setup and Access Control	29
10.4.2	Managing Agents, Packages, and LLM Providers	29
10.4.3	Managing Scheduler, Rules, and Alerts	29
10.5	Administrator Guide	29
10.6	Troubleshooting Guide	29

11 SYSTEM SNAPSHOTS	30
11.1 System Snapshots	30
12 FUTURE SCOPE AND ENHANCEMENTS	38
13 CONCLUSION	39
BIBLIOGRAPHY	39

1. INTRODUCTION

In the present era of digital transformation, organizations are collecting more endpoint and infrastructure telemetry than ever before, yet many teams still struggle to turn that raw data into usable security decisions. Large enterprises often deploy combinations of SIEM, EDR, and rule-based monitoring platforms, but smaller security teams, academic labs, and emerging SaaS providers are frequently left with fragmented tools that are difficult to operate, expensive to scale, or limited in multi-tenant flexibility.

ScropIDS addresses this gap by proposing a multi-tenant cloud-native intrusion detection platform designed to simplify the journey from endpoint telemetry to analyst action. Instead of forcing users to manually correlate raw logs from several hosts, the platform accepts normalized JSON events from endpoint agents, stores them in a tenant-scoped backend, aggregates them into meaningful windows, evaluates those windows using rules and optional large language model assistance, and then surfaces actionable alerts through a centralized web interface.

From an implementation perspective, ScropIDS is built as a full-stack distributed platform. The backend is implemented with Django and Django REST Framework, background scheduling is handled through Celery and Redis, persistent storage is managed using PostgreSQL, the dashboard is built with React + Vite + TypeScript, and the endpoint agent is written in Go for cross-platform packaging. The project therefore combines software engineering, security operations, and cloud-native deployment ideas within a single capstone system.

This report presents the full technical narrative of ScropIDS. It discusses the problem context, rationale, feasibility, software requirements, architecture, design diagrams, implementation approach, testing, deployment, maintenance, and future roadmap. It also preserves the academic report structure expected by the reference capstone template while replacing its content with the actual features and workflow of the present project.

2. PROFILE OF THE PROBLEM, RATIONALE & SCOPE OF THE STUDY

2.1 Profile of the Problem

Modern organizations generate an enormous amount of endpoint activity data from workstations, servers, and virtual machines. Suspicious process execution, repeated authentication failures, outbound beaconing, unusual file activity, and other warning signs often appear first at the endpoint level. However, in practice, this telemetry is usually scattered across local tools, inconsistent logs, and manual review processes.

In many small or medium-scale environments, analysts do not have a single structured system that can answer essential questions quickly. They may struggle to determine which endpoint is producing the most suspicious activity, whether similar signals are repeating across multiple hosts, or whether a particular stream of events is benign operational noise or a meaningful threat pattern. This leads to delayed triage, incomplete visibility, and a high likelihood of alert fatigue.

The problem becomes even more challenging in a multi-tenant environment. A shared platform must maintain strong tenant isolation while still offering central administration, secure agent enrollment, customizable detection policy, and usable analyst workflows. If tenancy is treated as an afterthought rather than a core design requirement, the risk of data leakage or operational confusion increases significantly.

Observation of existing practical workflows suggests several structural shortcomings:

1. **Lack of Unified Telemetry Intake:** Endpoint events are often distributed across scripts, host-native tools, or manually collected logs rather than normalized into a consistent pipeline.
2. **High Analyst Overhead:** Analysts are forced to inspect raw log records instead of consuming summarized, prioritized, and explainable signals.
3. **Poor Tenant-Aware Controls:** Shared systems frequently lack clear tenant boundaries, configurable organization scoping, or per-tenant onboarding workflows.
4. **Weak Operational Packaging:** Even when good detection logic exists, onboarding endpoint agents and synchronizing configuration remains cumbersome.
5. **Inconsistent Explainability:** Alerts may exist, but the reasoning and recommended next steps are often missing or too generic to be useful.
6. **Limited Adaptability:** Organizations with different budgets or privacy policies may want API-hosted AI models, local models, or no AI dependency at all, but existing workflows often support only one mode.

These shortcomings indicate a strong need for a platform that combines endpoint telemetry collection, secure ingestion, scheduler-controlled aggregation, rule-based escalation, optional LLM-assisted reasoning, and multi-tenant dashboard access within a single consistent operational model.

2.2 Rationale

The rationale behind ScropIDS is both practical and academic. Practically, the project addresses the recurring difficulty of building a usable intrusion detection workflow out

of scattered components. Many tools can collect logs, and many tools can display dashboards, but far fewer systems combine secure tenant onboarding, endpoint packaging, background aggregation, explainable triage, and configuration management in a single lightweight platform.

Academically, the project is valuable because it brings together multiple important domains of modern computing: systems programming through the Go agent, web application engineering through Django and React, asynchronous processing through Celery, data modeling through PostgreSQL, secure secret handling, multi-tenant access control, and machine-assisted decision support using strict JSON-based LLM outputs.

Another important rationale is flexibility. Some organizations may prefer OpenAI-compatible APIs for easier deployment, while others may require local inference through Ollama for cost, privacy, or offline operation. ScropIDS intentionally supports both. This makes the project more useful as a real engineering artifact rather than a narrow one-off demonstration.

The platform also reflects a usability-centered viewpoint. Intrusion detection is not only about generating alerts; it is about enabling analysts and administrators to understand the context of those alerts. By pairing aggregation windows, rule escalation, and human-readable reasoning, ScropIDS attempts to reduce the gap between raw telemetry and operational action.

2.3 Scope of the Study

The present study covers the design, implementation, and documentation of a production-oriented IDS platform with the following scope:

- Cross-platform endpoint agent onboarding and event submission.
- Tenant-aware backend design using organizations, memberships, and scoped data ownership.
- Secure enrollment using one-time tokens and organization access tokens.
- Event ingestion, normalization, persistence, and scheduled aggregation.
- Rule-assisted and LLM-assisted threat analysis using validated JSON outputs.
- Alert queue generation and dashboard-driven analyst workflows.
- Operational configuration pages for LLM providers, scheduler settings, and escalation rules.
- Docker Compose deployment for reproducible local or pilot-scale environments.

The study does not attempt to deliver a full commercial EDR suite with kernel-level telemetry, enterprise-scale policy engines, advanced threat intelligence fusion, or complete incident response orchestration. Instead, it focuses on building a strong and extensible foundation that demonstrates technical depth, operational relevance, and future viability.

2.4 Problem Statement

The core problem addressed by this project is the absence of a unified, multi-tenant, cloud-native platform that can securely enroll endpoint agents, collect structured telemetry, aggregate suspicious activity, produce explainable threat assessments, and present those assessments to users through a central web console.

ScropIDS solves this problem by combining secure agent-to-cloud communication, tenant-aware backend architecture, scheduler-driven event grouping, configurable rules, optional LLM analysis, and dashboard-based operations. The resulting platform is intended to reduce analyst overhead, improve explainability, and create a more practical IDS workflow for academic, pilot, and small-scale operational use.

3. EXISTING SYSTEM

3.1 Introduction

Before designing ScropIDS, it is important to understand how security monitoring is usually handled in real environments. Most existing setups are not completely wrong or useless; they solve parts of the problem. The difficulty is that they often work as separate pieces: one tool collects logs, another tool stores them, another dashboard shows alerts, and a human analyst still has to connect the meaning.

For example, a small company may have Windows Event Viewer on each system, a few Linux log files on servers, and some custom scripts that check suspicious commands. This gives basic visibility, but it does not create a single tenant-aware platform where agents can be enrolled, events can be grouped, alerts can be explained, and administrators can act from one dashboard. ScropIDS was designed around this practical gap.

3.2 Existing Software Analysis

The systems below are useful reference points for ScropIDS. They show what is already available and also explain why a focused capstone platform is still meaningful.

3.2.1 Wazuh / OSSEC

Wazuh and OSSEC are popular host-based intrusion detection systems. They can collect logs, monitor file integrity, apply rules, and generate alerts from endpoints. In a typi-

cal use case, an organization installs agents on machines and sends events to a central manager for review.

These systems are strong for traditional security monitoring, but they are not designed as a lightweight academic SaaS product out of the box. A student team or small prototype still needs to build custom tenant onboarding, dashboard flows, token handling, rule editing, and explainable alert views around them. In simple terms, they are powerful engines, but they do not directly provide the exact product experience ScropIDS is trying to demonstrate.

3.2.2 Splunk / Elastic Security

Splunk and Elastic Security are widely used for log ingestion, search, dashboards, and alerting. A security team can send large volumes of logs into these tools and build queries to detect suspicious activity. For example, an analyst may search for repeated failed logins, unusual PowerShell commands, or rare network connections across many hosts.

The limitation is cost and complexity. These platforms usually require careful pipeline setup, storage planning, dashboard configuration, and tuning. For a capstone project, the goal is not to recreate a full enterprise SIEM. The goal is to build a smaller system that clearly shows the complete workflow: agent enrollment, event ingestion, aggregation, rule evaluation, optional LLM-assisted explanation, and a dashboard for triage.

3.2.3 Commercial EDR Platforms

Commercial EDR products such as Microsoft Defender for Endpoint and CrowdStrike Falcon provide mature detection, threat intelligence, investigation timelines, and response features. In enterprise environments, these platforms are very useful because they combine deep endpoint telemetry with vendor-maintained detection logic.

However, they are mostly closed ecosystems. Their internal detection logic, data models, and platform behavior are not always transparent enough for academic explanation. They also depend on licenses and enterprise assumptions. ScropIDS is not positioned as a replacement for such products; it is a transparent implementation that shows how similar ideas can be engineered in a smaller, understandable, and extensible way.

3.2.4 Manual Log Review and Script-Based Monitoring

Many small teams still depend on manual checks. An administrator may inspect Linux `/var/log` files, Windows Event Viewer, firewall logs, or simple script outputs. This approach is easy to start because it does not require a large platform.

The weakness appears when the number of machines grows. For example, one endpoint

may show repeated failed logins, another may show suspicious process execution, and a third may show unusual outbound traffic. If these signals remain in separate files, the analyst may miss the larger pattern. ScropIDS improves this by collecting events in one backend, grouping them through scheduled aggregation, and presenting alerts with clearer context.

3.3 Summary of Gaps in Existing Systems

The existing systems prove that security monitoring is a mature area, but they also leave practical gaps for the target environment of this project. The main gaps are:

- **Fragmented workflow:** data collection, alert review, rule configuration, and explanation are often handled in different places.
- **Limited tenant focus:** many tools need additional engineering before they can support clean organization-wise isolation.
- **High setup burden:** full SIEM or EDR-style environments can be heavy for academic, pilot, and small-team use.
- **Low explainability:** alerts may show what happened, but not always why it matters or what action should be taken next.

Because of these gaps, ScropIDS focuses on a practical middle path: a transparent multi-tenant IDS platform with agent-based telemetry, scheduler-driven aggregation, rule-based detection, optional LLM assistance, and a dashboard designed for direct triage.

3.4 Data Flow Diagram: Existing System

The existing manual or loosely integrated monitoring workflow is illustrated in Figure 1. The figure emphasizes how endpoint activity is often reviewed only after being scattered into disconnected tools and informal analyst processes.

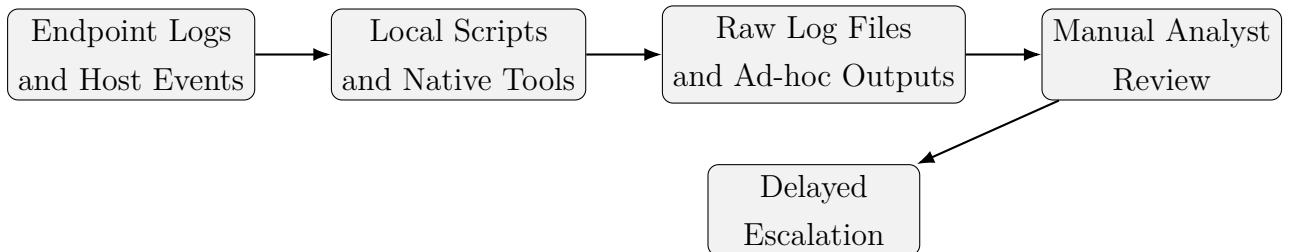


Figure 1: Existing system data flow in a fragmented monitoring workflow.

3.5 Innovations in the Proposed ScropIDS System

ScropIDS introduces several innovations relative to the existing baseline:

- **Tenant-aware platform design:** every core record is organization-scoped, supporting multi-tenant isolation from the start.
- **Structured onboarding:** both one-time enrollment tokens and organization access tokens are supported for safe agent registration.
- **Scheduler-driven aggregation:** raw events are grouped into analysis windows rather than being treated as isolated records only.
- **Hybrid triage model:** deterministic rules and validated LLM outputs cooperate rather than competing.
- **Cross-platform packaging:** downloadable artifacts are generated for multiple operating systems and architectures.
- **Integrated administration:** LLM configuration, scheduler settings, rules, alerts, and agent downloads are all available from the same platform.

Table 1 summarizes this comparison.

Category	Common Limitation	ScropIDS Improvement
Manual monitoring	Slow correlation and high analyst effort	Automated aggregation and alert queue
Traditional HIDS	Limited product-style tenant workflows	Built-in tenant administration and onboarding
SIEM platforms	Heavy operational overhead	Lightweight focused deployment model
Commercial EDR	Closed ecosystem and low transparency	Extensible academic-friendly architecture

Table 1: High-level comparison between existing approaches and ScropIDS.

4. PROBLEM ANALYSIS

4.1 Product Definition

ScropIDS is a geospatially independent, cloud-native, multi-tenant intrusion detection platform designed to receive endpoint telemetry from distributed agents, transform that telemetry into meaningful aggregate windows, and generate alerts that analysts can understand and act upon.

At the core of the product is a separation between three perspectives:

- **Endpoint perspective:** agents collect local host signals and submit them securely.
- **Platform perspective:** the backend stores, aggregates, analyzes, and prioritizes telemetry.
- **User perspective:** analysts and administrators interact with alerts, configurations, and onboarding workflows through the dashboard.

The product therefore sits at the intersection of endpoint monitoring, SaaS administration, and explainable threat analysis. It is not merely a log collector or a dashboard. It is an engineered workflow that connects enrollment, telemetry, aggregation, reasoning, and analyst action.

Table 2 identifies the major components of the system.

Component	Description
Endpoint Agent	Cross-platform Go binary for enrollment, heartbeat, event submission, and runtime sync
Secure Ingest API	Django REST endpoints for authenticated telemetry and tenant workflows
Tenant Model	Organization, membership, and scoped ownership model for data isolation
Aggregation Engine	Background scheduler logic that groups unprocessed events into windows
Rule Engine	Threshold and correlation-based escalation logic
LLM Layer	OpenAI-compatible or Ollama-backed analysis with strict JSON validation
Alert Queue	Tenant-scoped alert records with threat level, confidence, and reasoning
Web Dashboard	React interface for analysts and administrators

Table 2: Key components of the ScropIDS platform.

4.2 Feasibility Analysis

4.2.1 Technical Feasibility

The system is technically feasible because it is built with widely adopted technologies that are well matched to the problem. Django REST Framework provides a stable API layer, PostgreSQL offers robust relational storage with JSON support, Redis and Celery enable periodic background execution, React + Vite support a responsive frontend, and Go provides an efficient path for cross-platform agent compilation.

The repository already contains concrete implementations for organizations, agent authentication, event ingestion, aggregation windows, alert storage, scheduler profiles, LLM provider management, and dashboard pages. This means the project has moved beyond conceptual design into working system construction.

The modularity of the codebase also improves technical feasibility. The agent, backend, frontend, and documentation are clearly separated, which makes maintenance and extension more manageable.

4.2.2 Economic Feasibility

From an economic perspective, ScropIDS is viable because its primary technology stack is open source. A local deployment can be executed using community-supported runtimes without mandatory licensing costs. This is important for academic work and small pilot environments.

The platform also gives operators control over inference cost. Tenants can configure paid API-hosted providers or choose local inference through Ollama. That flexibility makes the solution more realistic for environments with budget constraints or data sovereignty concerns.

Operationally, the use of Docker Compose reduces setup friction and shortens the path from development to demonstration. This lowers the total cost of experimentation and adoption.

4.2.3 Operational Feasibility

Operational feasibility depends on whether real users can actually manage and benefit from the system. ScropIDS has been designed so that administrators can create organizations, rotate tokens, manage providers, edit scheduler rules, and review alerts from one dashboard instead of across several disconnected tools.

The agent enrollment experience is also operationally feasible because it supports both interactive setup and environment-variable-based execution. This allows it to fit classroom demos, test labs, or semi-automated rollout scenarios.

The presence of documented smoke-test flows further supports operational feasibility, because deployment verification can be repeated in a structured manner.

4.2.4 Time Feasibility

The project is time-feasible within a capstone schedule because the implementation can be approached in phases. The system architecture allows early progress on backend models and APIs, followed by dashboard workflows, agent onboarding, testing, and documentation.

This staged approach reduces dependency bottlenecks. For example, the dashboard can be built against a stable API contract while the agent packaging workflow is still being refined. Similarly, the scheduler and rule engine can function before advanced LLM tuning is complete.

Time feasibility is therefore improved by modular decomposition and by the clear separation between core functionality and future enhancements.

4.2.5 Legal and Ethical Feasibility

Intrusion detection inevitably involves the processing of sensitive system data. For that reason, any deployment of ScropIDS must be authorized by the organization operating the monitored endpoints. Consent, policy compliance, and data minimization are critical.

The design of ScropIDS supports legal and ethical feasibility by treating LLM output as advisory, encrypting provider secrets at rest, requiring authenticated agent communication, and preserving tenant boundaries. These measures do not eliminate all risk, but they demonstrate responsible engineering choices.

Because the project is intended for academic and controlled deployment contexts, ethical feasibility also depends on transparency. The platform should clearly communicate what data it collects, how it stores it, and how administrators can control access.

A practical deployment must also define responsibility before any endpoint agent is installed. Administrators should document which machines are included in the pilot, who can view collected telemetry, how long records are retained, and what action is taken when an alert is generated. This reduces uncertainty for users and makes the project easier to review from both technical and ethical perspectives.

Legal feasibility is also influenced by where model inference takes place. If a hosted LLM provider is used, the operating organization should review the provider’s retention policy, supported region, and contractual terms before any endpoint-derived summary is transmitted outside its environment. In deployments where this review is difficult or where policy requires tighter control, ScropIDS can operate with local inference or reduced analysis modes so that sensitive telemetry does not leave the authorized boundary.

Ethically, the system should remain focused on defensive monitoring rather than unchecked surveillance. That means administrators should define narrow collection goals, avoid collecting unrelated personal content, and periodically review whether each enabled data source is still necessary for threat detection. Governance over token rotation, membership assignment, rule tuning, and provider configuration is therefore not only a technical concern but also an accountability mechanism that helps ensure the platform is used in a proportionate and reviewable manner.

4.3 Project Plan

The project was organized as a staged engineering effort, shown in Table 3.

Phase	Primary Activities	Key Outcome
Phase 1	Problem analysis, literature study, and system scoping	Clear project direction
Phase 2	Tenancy model, architecture, and API contract design	Design foundation established
Phase 3	Backend implementation for organizations, agents, ingest, and alerts	Core service layer operational
Phase 4	Frontend implementation for overview, alerts, agents, rules, and scheduler	Usable operational dashboard
Phase 5	Go agent onboarding, packaging, and runtime profile synchronization	End-to-end telemetry workflow enabled
Phase 6	Testing, documentation, report preparation, and refinement	Submission-ready capstone deliverables

Table 3: Project plan used for the staged execution of ScropIDS.

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 Introduction

Software requirement analysis defines what the system must do, how different users interact with it, and what quality constraints it must satisfy. In ScropIDS, this analysis is particularly important because the platform supports both human users and machine agents. Human users need visibility, control, and safety; agents need a stable, secure, and low-friction communication pathway.

The requirement analysis also reflects the multi-tenant nature of the project. Requirements are not only about detection logic; they are also about isolation, configuration, explainability, and manageability.

5.2 General Description

The core actors in the system are summarized in Table 4.

Actor	Role in the System
Platform Administrator	Manages users, organizations, and operational setup at a broader level
Tenant Owner / Admin	Creates tokens, configures providers, tunes rules, and supervises tenant operations
Security Analyst	Reviews alerts, interprets reasoning, and updates alert status based on investigations
Endpoint Agent	Enrolls with backend, syncs runtime profile, sends events, and sends heartbeat signals

Table 4: Primary actors in the ScropIDS platform.

5.3 Specific Requirements

5.3.1 Functional Requirements

The functional requirements of ScropIDS are grouped below.

Tenant and User Management

- FR1: The system shall support creation of tenant organizations.
- FR2: The system shall maintain organization memberships with role-specific access.
- FR3: The system shall allow dashboard users to authenticate using session-based login.

Agent Enrollment and Endpoint Operations

- FR4: The system shall issue one-time enrollment tokens for controlled onboarding.
- FR5: The system shall provide organization access tokens for faster bulk-style enrollment.
- FR6: The system shall generate unique `agent_id` and `agent_token` values for enrolled endpoints.
- FR7: The agent shall send heartbeat messages and event batches to the backend.
- FR8: The agent shall retrieve runtime scheduler configuration from the backend.

Telemetry and Analysis

- FR9: The backend shall ingest normalized event JSON and store tenant-scoped records.
- FR10: The scheduler shall aggregate unprocessed events by organization and agent.
- FR11: The system shall support rule-based escalation over aggregate summaries.
- FR12: The system shall support LLM-assisted analysis using active tenant-specific providers.
- FR13: The platform shall create alerts when the resulting threat level crosses the configured threshold.

Dashboard and Administration

- FR14: The dashboard shall provide overview metrics including agents, events, and active alerts.
- FR15: The dashboard shall provide an alert queue with detailed reasoning and recommended actions.
- FR16: The dashboard shall provide an agents page with package downloads and token workflows.
- FR17: The dashboard shall provide pages for LLM provider configuration, scheduler tuning, and rule management.

5.3.2 Non-Functional Requirements

Performance

- NFR1: Routine dashboard queries should be responsive under ordinary tenant load.
- NFR2: Event batches should be accepted efficiently enough to support frequent agent submissions.
- NFR3: Scheduled aggregation should execute predictably based on the configured interval.

Security

- NFR4: Tenant records must remain isolated by organization ownership.
- NFR5: Agent authentication must be enforced on ingest and runtime config endpoints.

- NFR6: Provider API keys must be encrypted at rest.
- NFR7: LLM output must be validated before it is treated as an analysis result.

Reliability and Maintainability

- NFR8: The system should remain operable even if an LLM provider is unavailable.
- NFR9: The codebase should remain modular enough to support future collectors and rules.
- NFR10: The platform should be reproducible using documented setup steps and containerized services.

Usability

- NFR11: The UI should present alerts in a way that is understandable to analysts.
- NFR12: Administrative controls should be accessible from a centralized dashboard rather than scattered across separate tools.

6. DESIGN

6.1 System Architecture

The system architecture of ScropIDS is shown in Figure 2. It emphasizes the end-to-end flow from endpoints to analysts. The design is intentionally layered so that data collection, ingest, storage, background processing, model-assisted analysis, and user interaction can evolve independently while remaining operationally connected.

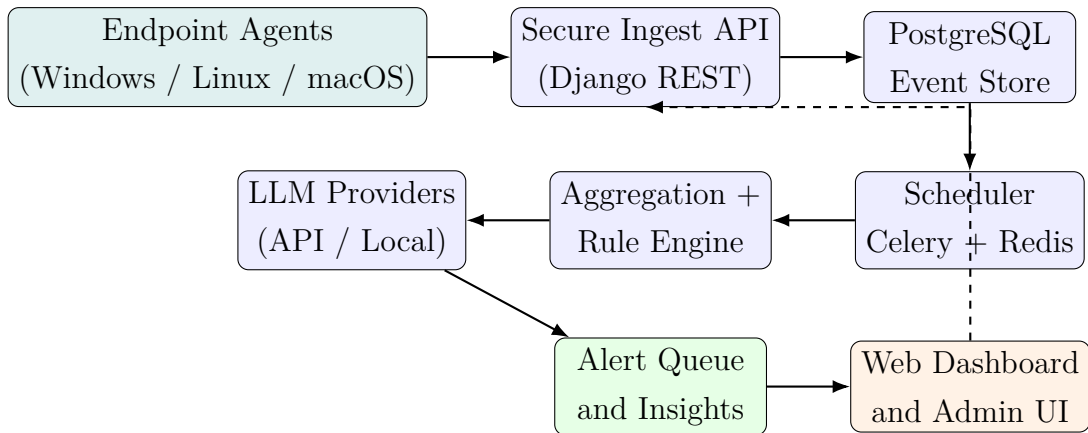


Figure 2: High-level system architecture of ScropIDS.

At the architectural level, the platform has four especially important characteristics:

- it separates endpoint behavior from backend processing,
- it keeps tenant ownership as a first-class data property,
- it treats scheduled aggregation as central rather than optional,
- it presents alerts to human users only after they have been structured and prioritized.

6.2 Design Notations

6.2.1 Data Flow Diagrams (DFD)

Figure 3 presents a context-level DFD in which the platform acts as a processing core between endpoint agents, tenant users, and persistent data stores. Figure 4 refines the internal flow to show ingestion, aggregation, rule/LLM analysis, and alert display.

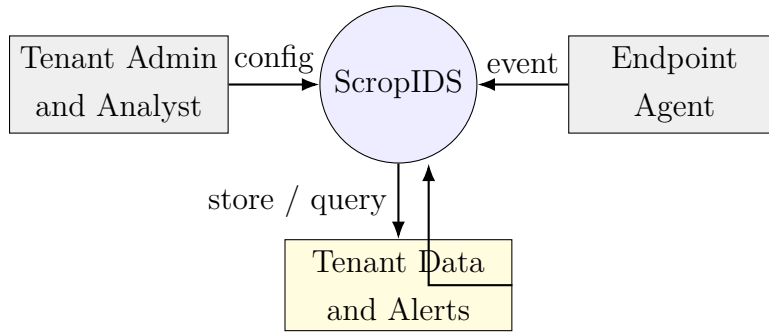


Figure 3: DFD level 0 for ScropIDS.

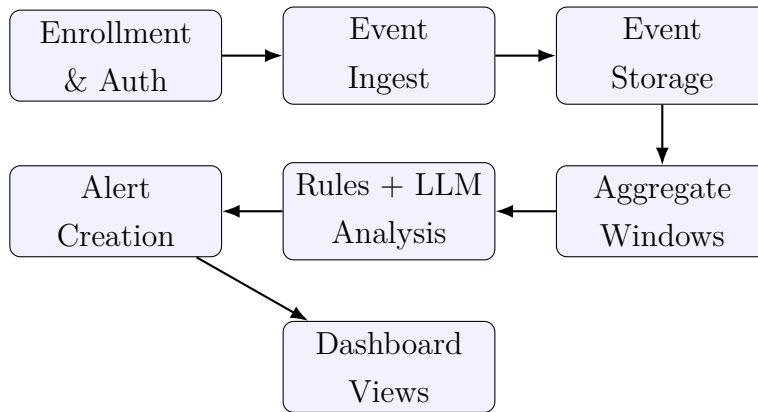


Figure 4: DFD level 1 showing the internal ScropIDS analysis flow.

6.2.2 Entity Relationship Diagram

The Entity Relationship Diagram in Figure 5 summarizes how tenant-aware data relationships are enforced throughout the system. The organization entity acts as the parent

scope for memberships, agents, events, aggregated windows, alerts, scheduler profiles, and LLM provider settings.

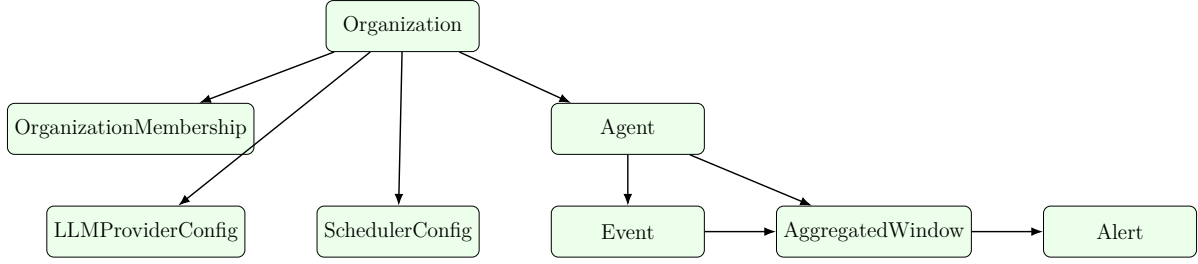


Figure 5: Simplified ER diagram of the ScropIDS data model.

6.3 Detailed Module Design

6.3.1 Agent Enrollment and Collection Module

This module is responsible for connecting endpoints to the platform. The Go agent supports interactive setup, direct credential mode, one-time enrollment token mode, and organization access token mode. After enrollment, it stores local configuration and periodically performs three tasks: refreshing runtime configuration, sending heartbeat messages, and transmitting event batches.

The collector design is intentionally modular. Current capabilities include process snapshots, security log sampling, network state snapshots, and system-status metadata. This layout leaves space for future file integrity and deeper operating-system-specific collectors without forcing a redesign of the enrollment logic.

6.3.2 Secure Ingest and Tenancy Module

The secure ingest module begins at the API boundary. It authenticates requests using agent headers, validates payload schemas, filters scaffold data where appropriate, and assigns every stored record to an organization. For human users, tenancy is resolved from session-authenticated membership data and an optional organization slug header.

This dual-authentication model is one of the strongest design features of the system because it recognizes that human and machine actors have fundamentally different operational identities. Analysts should not be authenticated like agents, and agents should not have the privileges of human users.

6.3.3 Aggregation, Rule Engine, and Threat Analysis Module

This module transforms raw telemetry into usable detection outputs. The scheduler identifies eligible unprocessed events, filters them based on configured minimum severity, groups them by organization and agent, and derives a summary payload. That payload includes counters such as failed logins, suspicious commands, new processes, and external

connections.

The summary is then passed through the analysis pipeline. If an LLM provider is active, the platform requests a strict JSON threat assessment. After that, the rule engine evaluates built-in and custom threshold logic, possibly escalating the final threat level or confidence value. This hybrid design means that LLMs enhance the system rather than acting as its sole detection logic.

6.3.4 Alerting, Dashboard, and Notification Module

Once a final analyzed state is available, the platform checks whether the threat level exceeds the configured alert threshold. If it does, a tenant-scoped alert record is created. Alerts are then displayed in the dashboard, where analysts can review details, change status, and use recommended actions as decision support.

This module also encompasses operator-oriented pages for rules, scheduler profiles, provider configuration, and agent downloads. In this way, the same product supports both analysis and administration without separating them into unrelated interfaces.

From a workflow perspective, the dashboard is designed to reduce time-to-triage rather than simply list raw findings. High-level widgets summarize current platform activity, while the alert queue provides a focused view of the items that need analyst attention first. Once an alert is opened, the user can inspect summarized evidence, confidence indicators, and recommended next steps without leaving the main investigation flow. This creates a more practical review experience than forcing analysts to jump between disconnected log views and configuration pages.

The notification aspect of the module is intentionally aligned with the lifecycle of an alert. A detection is first normalized into a durable record, then surfaced through dashboard status changes such as open, acknowledged, or resolved. This provides a basic but important operational loop in which analysts can distinguish new work from reviewed work. Even when external channels such as email or chat integrations are added later, the in-platform alert record remains the authoritative source of truth for state tracking and follow-up activity.

The same module also strengthens administrative observability. Operators can inspect rule behavior, adjust scheduler cadence, verify provider readiness, and distribute updated agent artifacts from within a shared interface layer. By concentrating these controls around the alert workflow, ScropIDS makes it easier to understand how configuration decisions influence the volume, quality, and urgency of generated detections.

6.4 Flowcharts

6.4.1 Agent Enrollment Flowchart

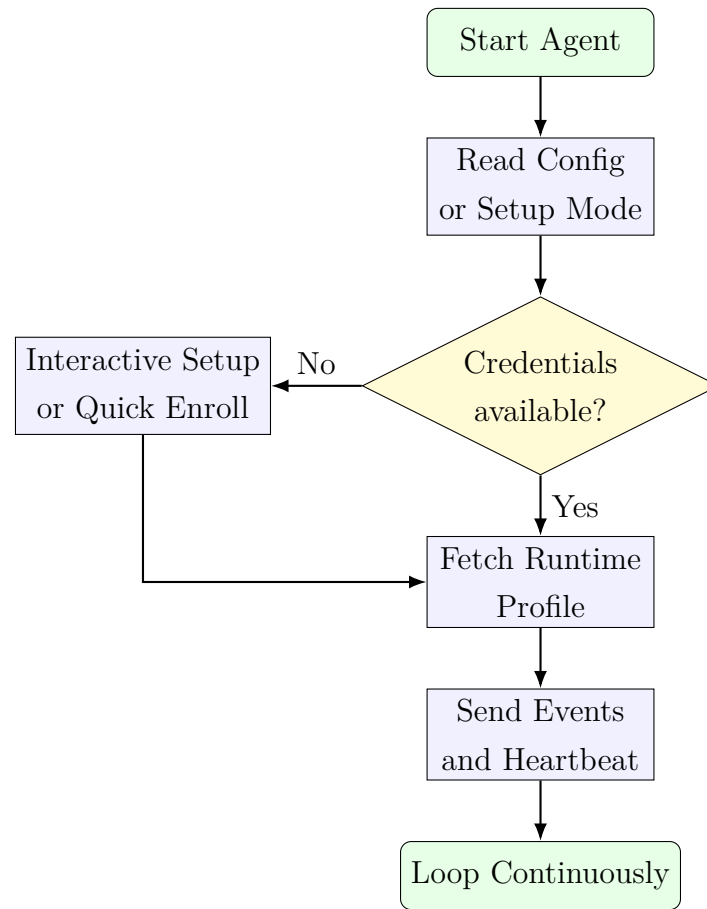


Figure 6: Flowchart for agent enrollment and runtime synchronization.

6.4.2 Alert Generation Flowchart

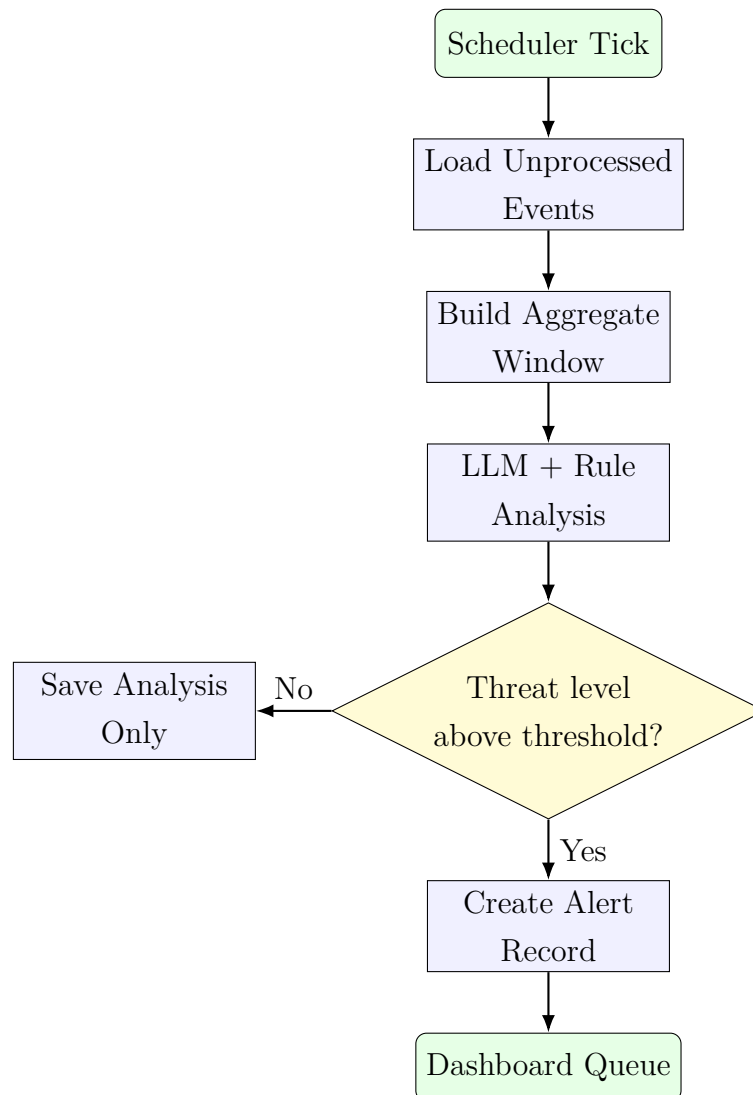


Figure 7: Flowchart for alert generation from aggregated telemetry.

6.5 Pseudocode

6.5.1 Aggregation Algorithm

```
for each active scheduler profile:
    determine the current aggregation window
    fetch unprocessed tenant events
    filter events below minimum severity
    group events by agent
    for each group:
        derive summary counters
        create aggregated window record
    mark included events as processed
    analyze new windows
```

6.5.2 Threat Escalation Algorithm

```
load llm_output and aggregate summary
set final_threat = llm_output.threat_level
set final_confidence = llm_output.confidence
for each enabled rule:
    if rule conditions match summary counters:
        final_threat = max(final_threat, rule.target_threat)
        final_confidence = max(final_confidence, rule.min_confidence)
        record rule match
if final_threat >= configured alert threshold:
    create alert
else:
    save analysis without alert creation
```

7. TESTING

7.1 Functional Testing

Functional testing focused on validating whether the major user-facing and agent-facing features behave as expected. Instead of checking only isolated functions, this level of testing verifies that the operational promise of the system is actually achieved.

For ScropIDS, this meant evaluating complete workflows rather than stopping at component-level correctness. A feature was considered functionally meaningful only when a user or agent could complete the intended path from start to finish. For example, it was not enough for the backend to accept an event payload in isolation; the same flow also needed to support agent enrollment, authenticated event submission, scheduler pickup, alert generation, and dashboard visibility in a way that matched the expected product behavior.

Functional testing also covered multi-tenant safety and administrative usability. The platform was checked to ensure that users could sign in, access the correct organization context, and view only the data associated with that tenant. Administrative paths such as provider configuration, token rotation, scheduler tuning, and artifact download were likewise treated as part of the functional surface because operational tools are essential to whether the IDS can be used effectively in practice.

On the agent side, testing emphasized realistic runtime behavior. The Go client needed to complete enrollment, persist local configuration, refresh runtime settings, transmit heartbeat data, and upload telemetry without breaking the tenant association created during onboarding. These checks helped confirm that the agent and cloud services behaved as one coordinated system rather than as separate pieces that only work under manual intervention.

Representative scenarios included creating a tenant, enrolling a new endpoint, generating sample telemetry, confirming aggregate analysis, reviewing the resulting alert in

the queue, and updating alert status through the dashboard. By structuring the tests around these end-to-end journeys, the project gained stronger evidence that its implemented features are usable in the same sequence in which a real operator would encounter them.

Table 5 summarizes the major functional test areas.

Test Area	Purpose	Current Evidence
Backend unit tests	Validate token handling and aggregate processing logic	<code>./backend/.venv/bin/python manage.py test apps.core</code>
Frontend build	Verify the React dashboard compiles into a production bundle	<code>npm run build</code>
Go agent build	Verify cross-platform starter agent source compiles correctly	<code>go build -o /tmp/scropids-agent-verify ./cmd/agent</code>
SaaS smoke workflow	Validate create-tenant, enroll-agent, ingest, aggregate, and overview flow	Documented in <code>docs/smoke-test-saas.md</code>
Alert queue workflow	Validate alert creation, viewing, and status transitions	Verified through dashboard pages and API routes

Table 5: Functional testing summary for ScropIDS.

7.2 Structural Testing (White-Box Testing)

Structural testing was used to evaluate internal correctness rather than only visible outcomes. This was especially important in parts of the platform where boundary behavior or conditional branching affects security and reliability.

The main components emphasized in structural testing were:

- agent token round-trip validation,
- one-time token usability and expiry semantics,
- event aggregation and processed-state updates,
- rule threshold evaluation under edge conditions,
- JSON validation of LLM outputs,
- runtime configuration refresh behavior in the agent.

Component	Testing Approach	Outcome
Agent token model methods	Valid and invalid token comparisons	Only correct tokens authenticate
Enrollment token lifecycle	Fresh, expired, and used-token scenarios	Single-use and expiry behavior preserved
Aggregation pipeline	Event insertion followed by aggregation window creation	Events marked processed after inclusion
Rule escalation logic	Controlled summary counters around thresholds	Escalation occurs only when conditions match
LLM schema validation	Missing keys, wrong types, and valid JSON payloads	Invalid output rejected, valid output accepted

Table 6: Structural testing summary.

7.3 Levels of Testing

7.3.1 Unit Testing

Unit testing verifies small isolated parts of the system. In the backend, this includes model methods, token checking behavior, and the aggregate creation process. These tests are especially important because they confirm correctness before components are wired together into full workflows.

For frontend components, the codebase structure also supports isolated rendering and view-level testing in future iterations. Although the current focus is on build validation and integrated behavior, the component design supports deeper UI unit testing later.

7.3.2 Integration Testing

Integration testing ensures that separate modules exchange data correctly. In ScropIDS, this includes the path from authentication to organization resolution, the path from agent enrollment to ingest permissions, the path from event storage to aggregation, and the path from analysis output to alert visibility in the dashboard.

The documented smoke test flow is especially useful here because it exercises the system across several module boundaries using realistic API requests instead of only internal test doubles.

7.3.3 System Testing

System testing treats the platform as a complete operational product. For ScropIDS, system testing includes starting the services through Docker Compose, accessing the dashboard through the browser, running authenticated API calls, triggering background analysis, and reviewing the resulting alert queue.

This level of testing validates not only code behavior, but also service topology, environment configuration, cross-service dependencies, and the user-visible result of the overall design.

7.3.4 User Acceptance Testing (UAT)

User acceptance testing is concerned with whether the platform is usable and understandable to its intended audience. For ScropIDS, the primary UAT scenarios are:

- signing in and switching tenant context,
- understanding dashboard metrics,
- reading alert reasoning,
- downloading and onboarding agents,
- configuring providers and scheduler settings without confusion.

The main acceptance criterion is not just whether the system is functional, but whether users can confidently navigate it and interpret the alert outputs it generates.

7.3.5 Performance Testing

Although the current project phase emphasizes correctness and architecture more than large-scale benchmarking, performance remains a non-functional concern. The design anticipates horizontal scaling by separating ingestion, persistent storage, and background execution. Future performance testing will need to measure ingest throughput, scheduler latency, queue backlog behavior, dashboard responsiveness, and tenant isolation under concurrent load.

At the present stage, performance assurance comes primarily from architectural choices rather than full benchmark campaigns. Even so, the modular service topology already places the system on a reasonable path toward more advanced load testing.

8. IMPLEMENTATION

8.1 Implementation of the Project

The project is implemented as a monorepo with separate but coordinated backend, frontend, agent, and documentation areas. This structure supports parallel development while keeping the overall platform reproducible and consistent.

The backend is implemented in Django and Django REST Framework. It provides the data model, tenancy layer, agent authentication, organization workflows, ingest endpoints, alert APIs, and configuration APIs. Celery workers and beat processes are used to execute scheduler-controlled background analysis tasks.

The frontend is implemented using React + Vite + TypeScript. It contains the operational screens used by analysts and administrators, including overview metrics, alerts, agents, scheduler configuration, LLM provider management, and rules.

The endpoint agent is implemented in Go. It supports interactive setup, configuration persistence, quick enrollment, runtime schedule synchronization, and event submission. It can be packaged into multiple platform-specific artifacts, which are then exposed back to users through downloadable manifests.

Figure 8 shows the implementation topology as deployed via Docker Compose.

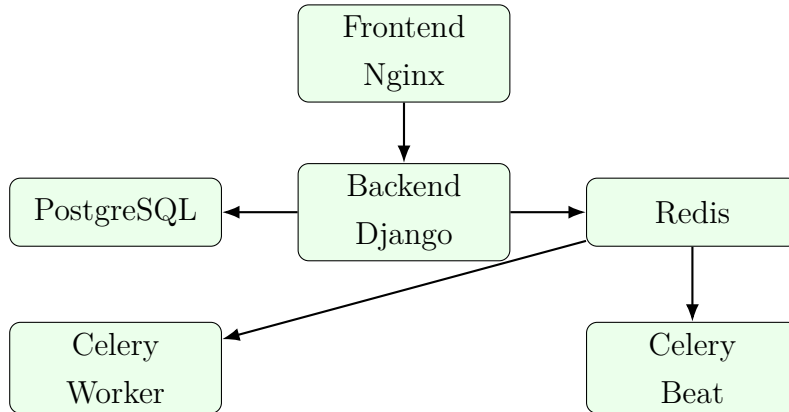


Figure 8: Service topology used during implementation and deployment.

8.2 Conversion Plan

The deployment of ScropIDS into a real environment is best approached in phases. A phased conversion reduces risk, allows operator feedback to be collected early, and makes it easier to verify that tenant controls, agent enrollment, and analysis workflows behave correctly before a larger rollout.

Step	Description
1	Prepare environment variables, database credentials, Redis connectivity, and backend secrets.
2	Start the services and validate health checks for database, backend, worker, and frontend.
3	Create an administrative user and at least one organization.
4	Configure an LLM provider or deliberately validate the fallback no-provider path.
5	Generate an enrollment token or rotate an organization access token.
6	Enroll pilot agents and validate heartbeat plus event acceptance.
7	Run scheduler ticks, review alert generation, and collect analyst feedback.
8	Expand to broader endpoint coverage only after pilot validation is stable.

Table 7: Phased conversion plan for ScropIDS deployment.

8.3 Post-Implementation and Software Maintenance

After deployment, the platform must be maintained across several dimensions:

- **Corrective Maintenance:** resolving bugs in authentication, ingest, scheduler behavior, or dashboard workflows.
- **Adaptive Maintenance:** updating external integrations such as model providers, packages, or infrastructure versions.
- **Perfective Maintenance:** improving UI clarity, detection quality, and performance based on operator feedback.
- **Preventive Maintenance:** patching dependencies, verifying backups, rotating tokens, and reviewing security posture proactively.

The maintenance cycle is illustrated in Figure 9.

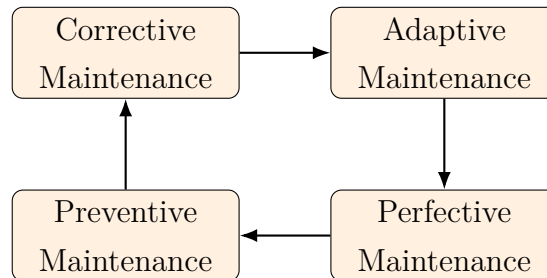


Figure 9: Maintenance cycle for the ongoing evolution of ScropIDS.

9. PROJECT LEGACY

9.1 Current Status of the Project

At the current stage, ScropIDS exists as a working capstone implementation rather than only a design proposal. The repository contains the backend data model, API routes, dashboard pages, agent starter, packaging scripts, and supporting documentation required to demonstrate the core workflow.

This means the project already supports a meaningful end-to-end scenario: create a tenant, enroll an agent, send telemetry, run aggregation, analyze an aggregate, create an alert, and review the result through a web interface. That level of completeness is an important marker of project maturity.

9.2 Remaining Areas of Concern

Despite its current maturity, several areas remain open for future hardening and refinement:

- deeper OS-specific telemetry collectors are still needed,
- more extensive load and endurance testing should be conducted,
- role-based access control can be refined beyond the present baseline,
- threat intelligence enrichment and richer mapping to security frameworks remain future enhancements,
- long-term audit and observability features can be expanded further.

These concerns do not invalidate the present implementation; rather, they define the next layer of work needed to transition from a strong capstone system to a more production-ready security product.

9.3 Technical and Managerial Lessons Learnt

9.3.1 Technical Lessons

Several technical lessons emerged from this project. First, strict data contracts are extremely valuable: once telemetry is normalized and model output is forced into a validated JSON schema, downstream processing becomes much more reliable. Second, deterministic rules and model reasoning are stronger together than either approach alone; rules provide consistency, while model output improves explanation and prioritization.

Third, tenant isolation must be engineered into the data model at the beginning of the project rather than bolted on later. Finally, deployment and packaging are not secondary

concerns. A security system that cannot be onboarded cleanly will struggle even if its detection logic is strong.

9.3.2 Managerial Lessons

From a managerial perspective, the project reinforced the importance of early requirement clarity and staged execution. Dividing the work into architecture, backend, frontend, agent, and validation phases reduced confusion and allowed progress to continue even when one component was still being refined.

The work also demonstrated the importance of documentation as part of development rather than after development. Architecture notes, API contracts, smoke test instructions, and report drafts made the project easier to validate and easier to explain.

Finally, iterative feedback was valuable. Small usability observations about the dashboard, token workflows, and wording of alerts had a large effect on the clarity of the final system.

10. USER MANUAL (HELP GUIDE)

10.1 Introduction

This section describes how intended users interact with ScropIDS. The platform serves three broad kinds of users: analysts who investigate alerts, tenant administrators who configure and onboard, and platform-level administrators who manage broader deployment concerns. It also serves endpoint agents, which function as authenticated machine actors.

10.2 System Requirements

Component	Minimum Requirement
Backend Host	2 vCPU, 4 GB RAM for pilot-scale deployment
Database	PostgreSQL 13+ with JSON-capable storage
Queue	Redis 6+
Agent Endpoint	Network access to backend and sufficient permission to read host signals
Frontend	Modern browser such as Chromium, Firefox, or Safari

Table 8: Baseline system requirements for ScropIDS.

10.3 Dashboard / Analyst User Guide

10.3.1 Account Registration and Login

Open the application and navigate to the login page. If the deployment supports self-registration, a user can create an account and initial organization through the registration flow. Otherwise, the user should log in using the credentials provided by the administrator.

After authentication, the top header allows the user to select the active tenant workspace. This step is important because all subsequent dashboard views, alerts, and configuration screens are scoped to the currently selected organization.

10.3.2 Using the Overview and Alerts Workspace

Once logged in, the user should begin on the Overview page. This screen provides a summary of total agents, recent event activity, active alerts, threat distribution, and other tenant-specific operational indicators.

To investigate suspicious activity, the user can open the Alerts page. Each alert displays threat level, confidence, host context, reasoning, and recommended action. The alert detail drawer gives a more readable explanation of why the system prioritized that event window.

10.3.3 Downloading, Enrolling, and Monitoring Agents

The Agents page provides three important capabilities. First, it shows the enrolled endpoints for the active tenant along with last-seen information. Second, it allows the administrator to rotate or copy the organization access token. Third, it exposes downloadable agent packages for multiple platforms and architectures.

After downloading the correct artifact, the operator can run the agent using interactive setup or environment variables. Once enrollment is complete, the endpoint should begin appearing in the dashboard and its heartbeat should update the last-seen field.

10.3.4 Handling Alerts and Recommended Actions

When reviewing alerts, users should treat the displayed reasoning and recommended action as guided decision support. The platform helps prioritize and explain signals, but final investigation and remediation decisions remain the responsibility of the human analyst.

Alerts can be moved through statuses such as **open**, **in_progress**, and **resolved**. This helps teams distinguish newly raised issues from actively investigated or closed items.

10.4 Tenant Administrator Guide

10.4.1 Organization Setup and Access Control

Tenant administrators are responsible for ensuring that the correct users belong to the correct organization. This includes validating memberships, confirming role assignments, and making sure the active workspace in the dashboard matches the intended tenant before administrative actions are taken.

10.4.2 Managing Agents, Packages, and LLM Providers

Tenant administrators can rotate organization access tokens, distribute agent packages, and enroll endpoints. They also manage LLM providers through the LLM Config page, where each provider can be activated, edited, or removed. The ability to switch between API-hosted and local inference is especially useful during pilot operation and testing.

10.4.3 Managing Scheduler, Rules, and Alerts

The Scheduler page controls aggregation interval, minimum severity, alert threshold, and agent runtime collector settings. The Rules page controls built-in and custom escalation behavior. Administrators should tune these carefully, because they directly influence how noisy or conservative the alert queue becomes.

10.5 Administrator Guide

Platform administrators operate at a broader scope than tenant administrators. Their responsibilities include deployment readiness, account provisioning, infrastructure verification, health monitoring, and broader policy oversight. In classroom or project demonstration environments, this role may be combined with the mentor or system operator role.

An administrator should regularly confirm:

- backend health endpoint availability,
- database and Redis connectivity,
- worker and beat service activity,
- integrity of environment variables and secrets,
- successful execution of migrations and static collection.

10.6 Troubleshooting Guide

- If agent enrollment fails, verify the organization slug, token value, and API base URL.

- If the dashboard is empty, confirm the correct tenant is selected and the user belongs to that organization.
- If no alerts are generated, check scheduler status, provider configuration, and alert threshold rules.
- If runtime sync fails, validate agent credentials and the `/api/v1/ingest/config/` endpoint.
- If the frontend does not load expected data, confirm backend CORS/CSRF settings and session authentication status.

11. SYSTEM SNAPSHOTS

11.1 System Snapshots

This section provides visual evidence of the ScropIDS platform implementation, covering the user journey from authentication to advanced threat triage and system administration.

The snapshots are arranged to reflect the main operational progression of the platform. The early figures focus on secure access and tenant onboarding, showing how a new organization enters the system and reaches the monitoring workspace. The middle figures highlight dashboard visibility, alert prioritization, and analyst review surfaces, which together demonstrate how telemetry is transformed into a readable threat narrative. The later figures move into administrative capability, including agent management, provider setup, scheduler tuning, and deployment verification.

Including these visuals is important because a security platform cannot be evaluated only through architecture diagrams or source code descriptions. A working intrusion detection system must also be understandable at the interface level: users should be able to identify where alerts appear, how severity is communicated, which controls are available to administrators, and how endpoint deployment is supported. The screenshots therefore complement the testing and design chapters by showing the product as it appears in actual use.

Taken together, the pages in this section demonstrate continuity between backend processing and user-facing experience. They show that ScropIDS is not merely a collection of APIs, but a complete operational workflow with login, telemetry review, explainable alert triage, configuration surfaces, and packaged deployment artifacts. This makes the section useful as implementation evidence while leaving the individual snapshot content unchanged.

Snapshot 1: Login Page

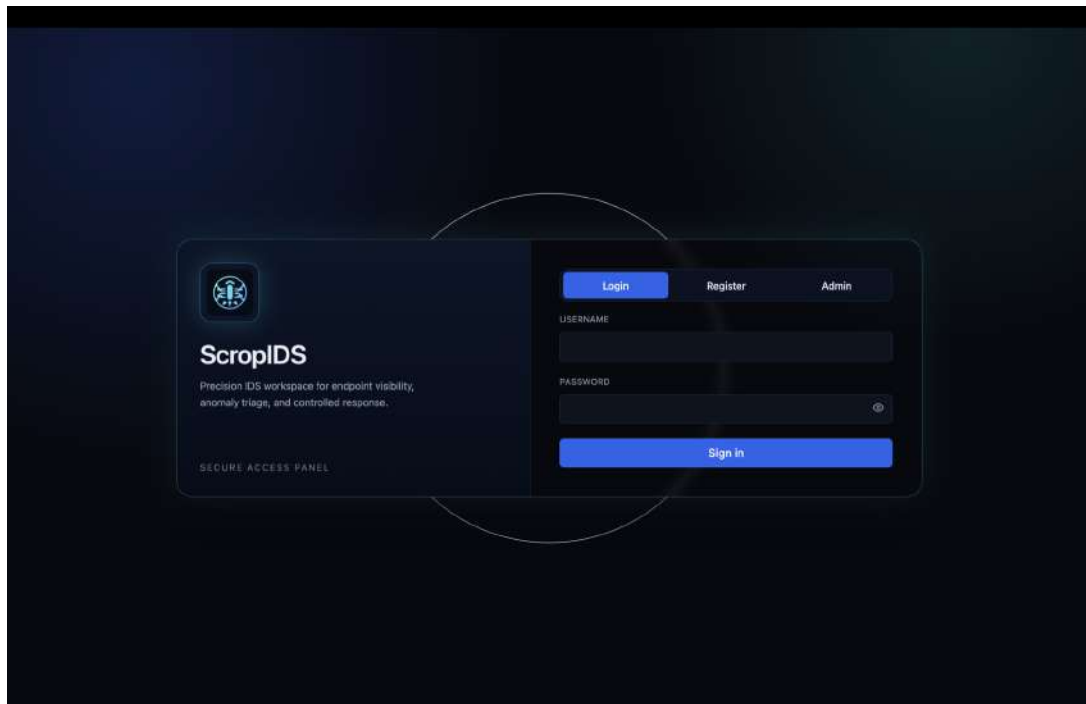


Figure 10: The secure access panel for user authentication.

Snapshot 2: Workspace Registration

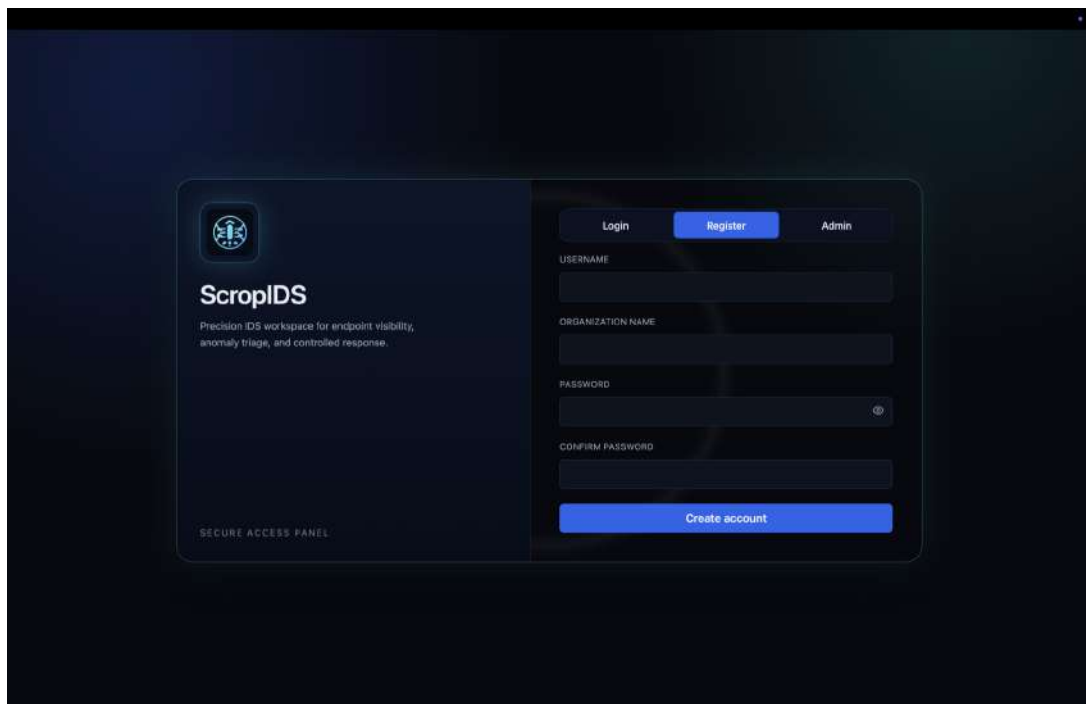


Figure 11: Multi-tenant registration flow for creating a new organization.

Snapshot 3: Dashboard Overview

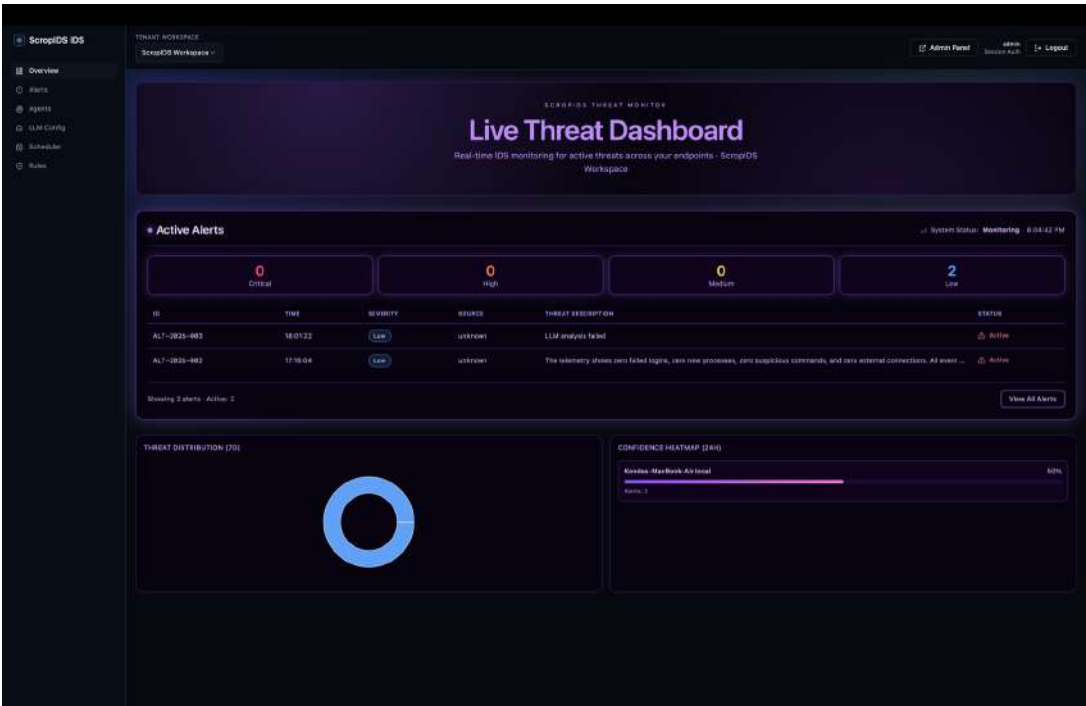


Figure 12: The live threat dashboard showing real-time monitoring status.

Snapshot 4: KPI Cards

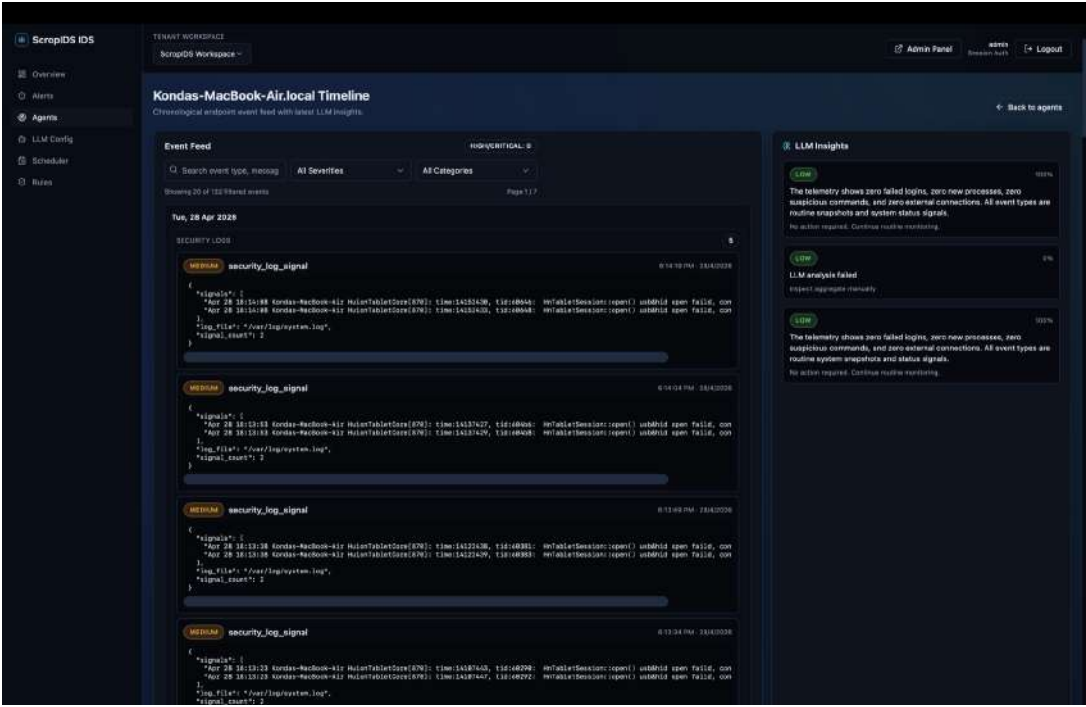


Figure 13: Operational metrics showing high-level security indicators.

Snapshot 5: Threat Distribution Chart

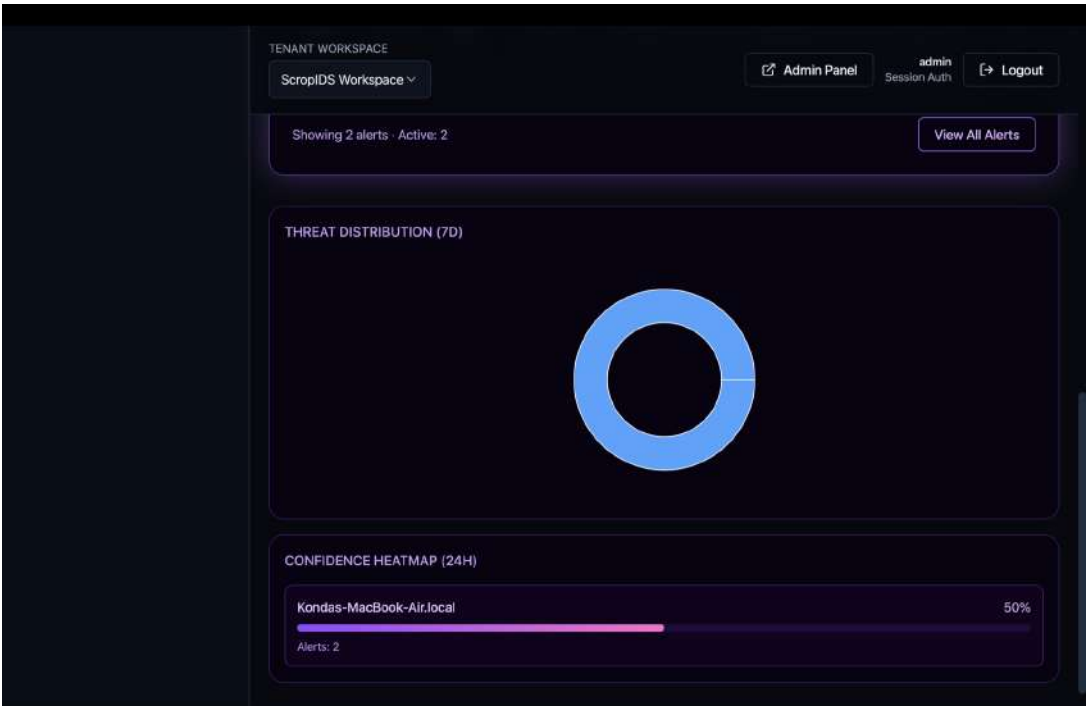


Figure 14: Visual breakdown of threat severity and confidence heatmaps.

Snapshot 6: Recent Alerts Table

The screenshot shows a section titled 'Active Alerts' with a 'System Status: Monitoring' indicator and the time '6:07:32 PM'. Below the title are four summary boxes for alert severity: Critical (0), High (0), Medium (0), and Low (2). The main part of the image is a table of active alerts.

ID	TIME	SEVERITY	SOURCE	THREAT DESCRIPTION	STATUS
ALT-2026-003	18:01:22	Low	unknown	LLM analysis failed	Active
ALT-2026-002	17:16:04	Low	unknown	The telemetry shows zero failed logins, zero new processes, zero suspi...	Active

At the bottom, it says 'Showing 2 alerts · Active: 2' and includes a 'View All Alerts' button.

Figure 15: Active alert queue with ID, severity, and threat descriptions.

Snapshot 7: Alerts Queue Page

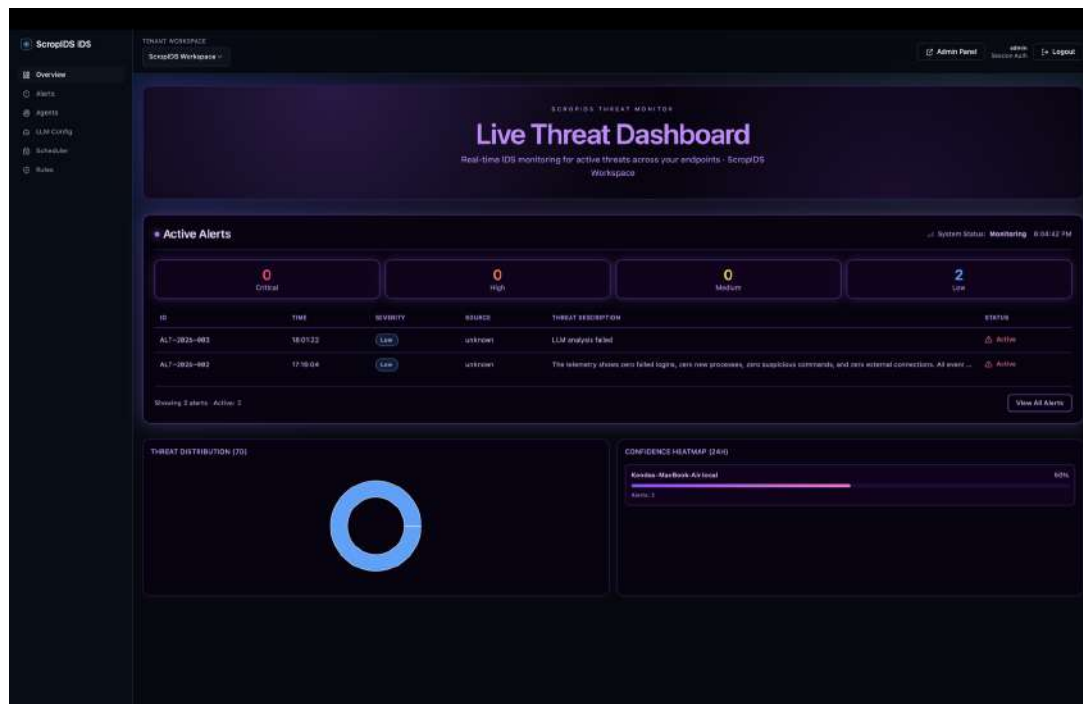


Figure 16: The centralized triage queue for security analysts.

Snapshot 8: Alert Detail Drawer

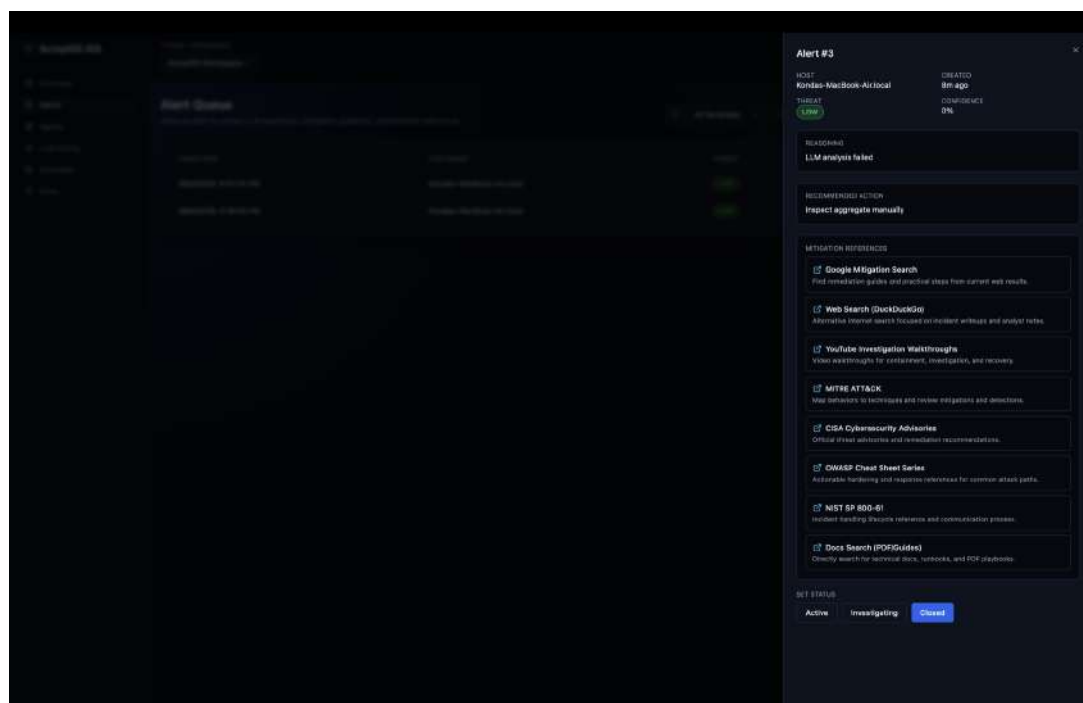
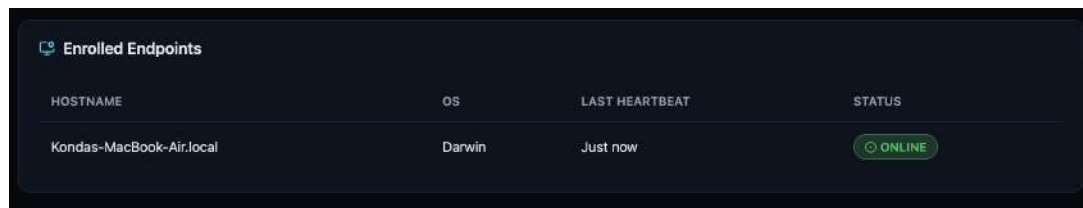


Figure 17: Detailed LLM insights providing reasoning and recommended actions.

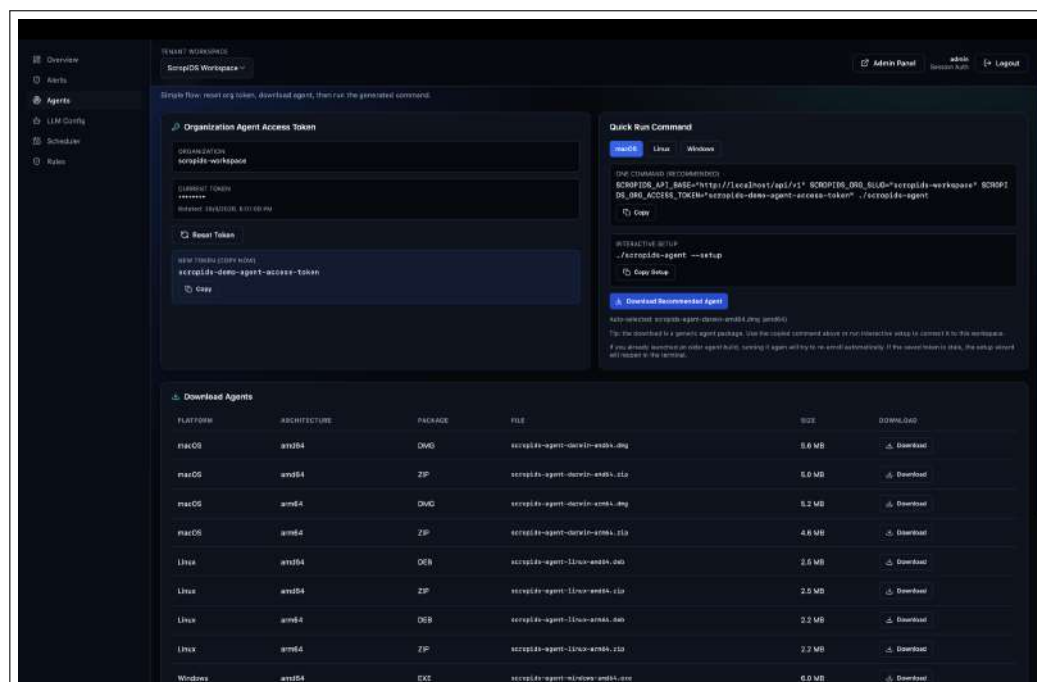
Snapshot 9: Agents Page



HOSTNAME	OS	LAST HEARTBEAT	STATUS
Kondas-MacBook-Air.local	Darwin	Just now	ONLINE

Figure 18: List of enrolled endpoints with real-time heartbeat and OS status.

Snapshot 10: Token Rotation Panel



Organization Agent Access Token

Current Token: refresh

Refresh Token: refresh

Quick Run Command

```
curl -X POST -H "Content-Type: application/json" -d '{"api_base": "http://localhost:8080", "org_id": "scriptide-workspace", "scriptide_org_access_token": "scriptide-org-access-token"}' http://localhost:8080/api/v1/scriptide-agent
```

Download Agents

PLATFORM	ARCHITECTURE	PACKAGE	FILE	SIZE	DOWNLOAD
macOS	amd64	DMG	scriptide-agent-darwin-amd64.dmg	5.6 MB	Download
macOS	amd64	ZIP	scriptide-agent-darwin-amd64.zip	5.0 MB	Download
macOS	arm64	DMG	scriptide-agent-darwin-arm64.dmg	5.2 MB	Download
macOS	arm64	ZIP	scriptide-agent-darwin-arm64.zip	4.6 MB	Download
Linux	amd64	DEB	scriptide-agent-linux-amd64.deb	2.6 MB	Download
Linux	amd64	ZIP	scriptide-agent-linux-amd64.zip	2.5 MB	Download
Linux	arm64	DEB	scriptide-agent-linux-arm64.deb	3.2 MB	Download
Linux	arm64	ZIP	scriptide-agent-linux-arm64.zip	2.7 MB	Download
Windows	amd64	EXE	scriptide-agent-windows-amd64.exe	6.0 MB	Download

Figure 19: Administrative interface for managing organization access tokens.

Snapshot 11: Artifact Download Section

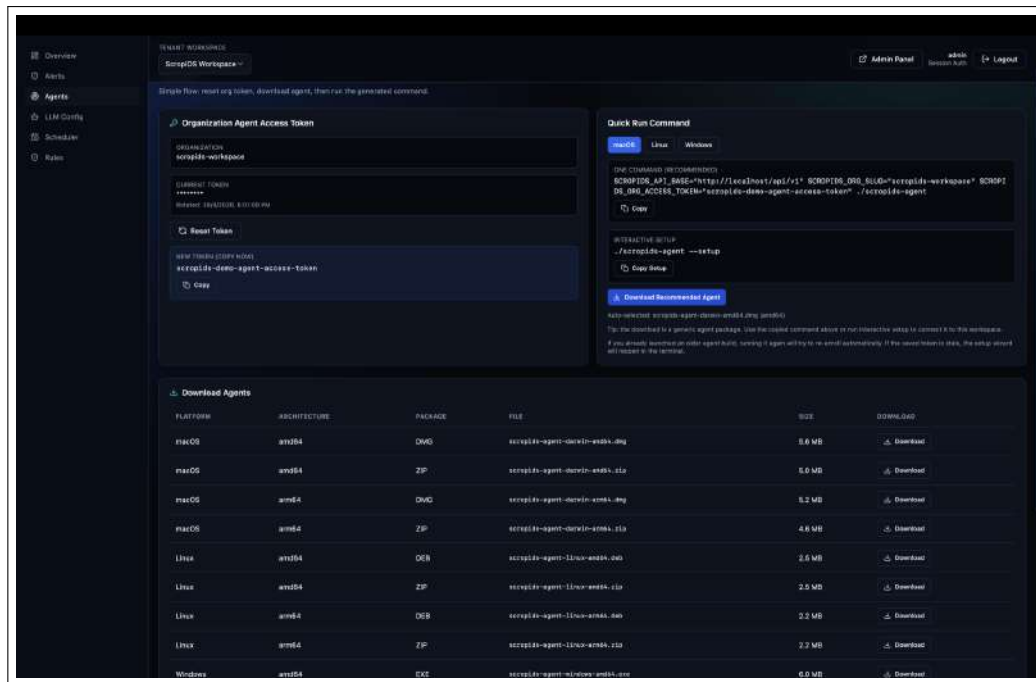


Figure 20: Cross-platform agent download manifests for Go-based binaries.

Snapshot 12: Quick Run Command

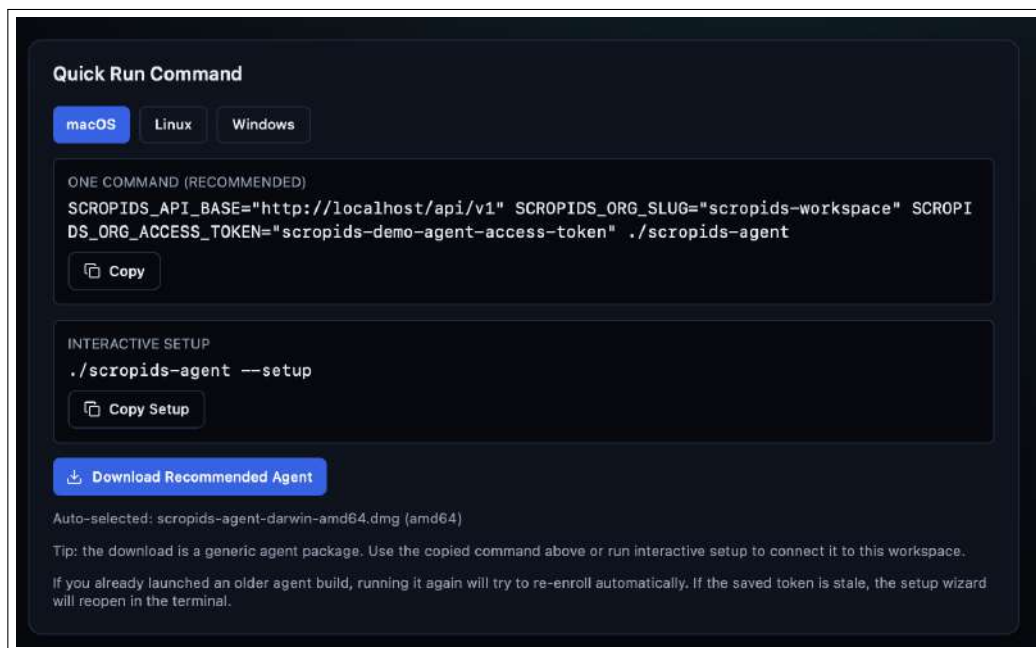


Figure 21: CLI-based agent enrollment and runtime execution.

Snapshot 13: Agent Timeline Page

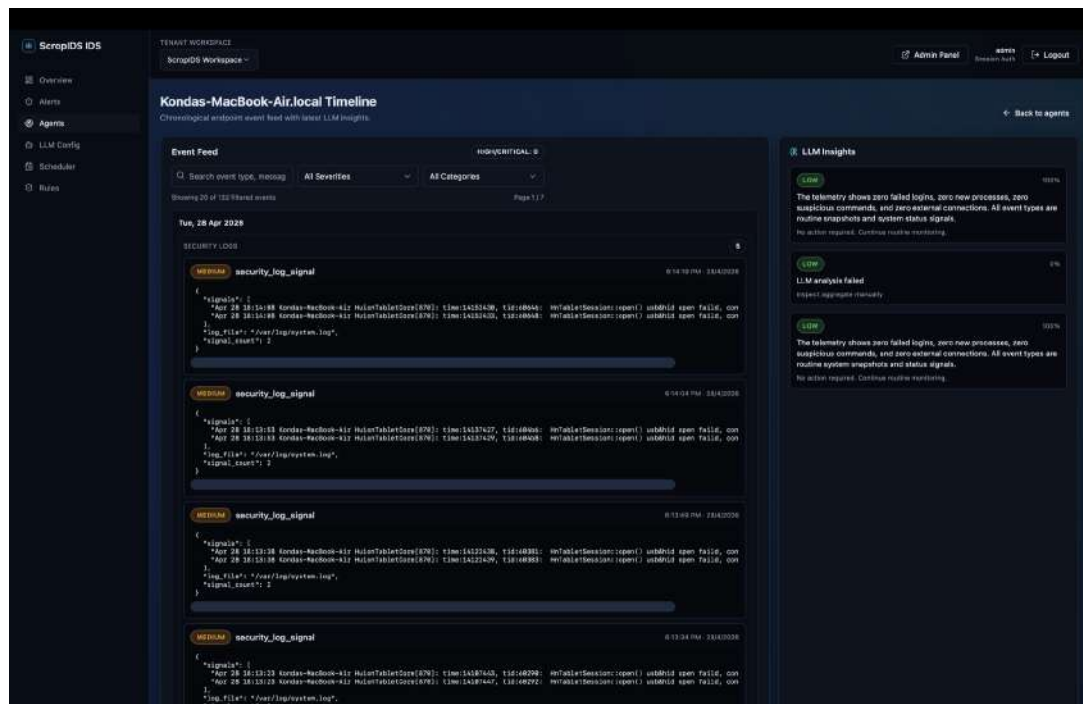


Figure 22: Chronological endpoint event feed with integrated LLM analysis.

Snapshot 14: LLM Provider Management

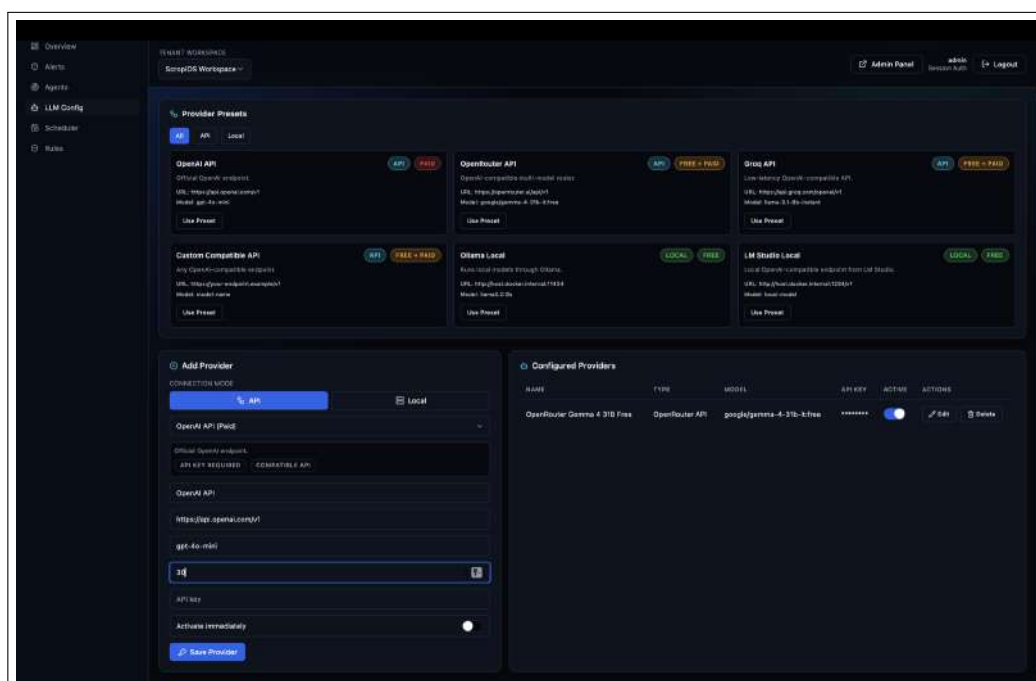


Figure 23: Configuration panel for OpenAI-compatible or local Ollama providers.

Snapshot 15: Scheduler and Rule Engine

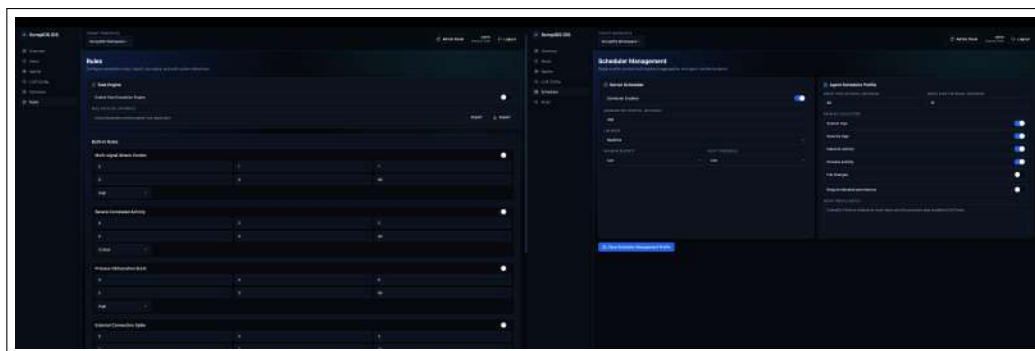


Figure 24: Fine-grained controls for aggregation windows and escalation rules.

Snapshot 16: Final Deployment Proof



Figure 25: Deployment verification showing running services and health check logs.

12. FUTURE SCOPE AND ENHANCEMENTS

ScropIDS already establishes a strong functional foundation, but several improvements can significantly enhance its long-term operational value.

- Implement richer OS-specific collectors for Windows event logs, Linux audit sources, and macOS unified logs.
- Introduce Sigma-compatible rule import and stronger mapping to MITRE ATT&CK techniques.
- Add threat intelligence enrichment for suspicious IPs, domains, and hash values.
- Improve role-based access control with more granular per-user and per-workspace policy.
- Extend alerting to richer outbound channels such as email, Slack, and Telegram in production-ready form.
- Support more advanced dashboard visualizations for investigation timelines and analyst collaboration.
- Add large-scale load testing, observability metrics, and orchestration support for Kubernetes or similar environments.

These enhancements are realistic next steps because the present architecture is already modular. The most important future work will likely focus on collector maturity, performance hardening, and deeper detection sophistication rather than rethinking the entire platform.

13. CONCLUSION

ScropIDS demonstrates that a capstone project can evolve into a meaningful and technically grounded intrusion detection platform when the design is approached holistically. Instead of stopping at raw log collection or a static dashboard, the system connects secure agent onboarding, telemetry ingestion, scheduler-driven aggregation, rule-assisted and LLM-assisted threat triage, and analyst-facing review workflows within a single operational product.

The platform also shows that multi-tenant security tooling can be designed explicitly rather than assumed indirectly. Tenant-aware ownership, scoped configuration, secure tokens, and organization-based workflows are all treated as primary design concerns, which improves both safety and clarity.

Most importantly, the project balances technical depth with practical usability. By providing explainable alerts, configurable provider modes, downloadable agents, and dashboard-based operations, ScropIDS goes beyond theory and into a deployable engineering artifact. With future improvements in collector maturity, enrichment, and scalability, the system can grow into an even more capable security operations platform.

BIBLIOGRAPHY

1. ScropIDS project repository documentation: `README.md`, `docs/architecture.md`, `docs/api-contract.md`, and `docs/smoke-test-saas.md`.
2. Django Documentation. <https://docs.djangoproject.com/>
3. React Documentation. <https://react.dev/>
4. Celery Documentation. <https://docs.celeryq.dev/>
5. PostgreSQL Documentation. <https://www.postgresql.org/docs/>
6. Docker Documentation. <https://docs.docker.com/>
7. MITRE ATT&CK. <https://attack.mitre.org/>
8. NIST Cybersecurity Framework 2.0. <https://www.nist.gov/cyberframework>